TP 2 - Connaissances en logique des propositions

L'objectif de ce TP est de construire une base de connaissances pour résoudre des énigmes logiques et d'implémenter l'algorithme de la résolution pour démontrer l'implication sémantique.

1 Découverte du code fourni

Un codage des propositions logiques vous est fourni dans le programme logic.py. Vous y trouverez des classes représentant les différents opérateurs de la logique des propositions, permettant par exemple d'exprimer une expression $(A \vee B) \wedge \neg C$ sous la forme :

```
And(Or(Symbol("A"), Symbol("B")),Not(Symbol("C"))).
```

Sur les deux méthodes présentées en cours pour démontrer une implémentation sémantique $KB \models \alpha$, seule la première, la vérification de modèle, est entièrement mise en œuvre dans la fonction `model_check()`.

Testez l'utilisation de ce programme avec l'option par défaut `-c` de clue.py, qui code un état du jeu Cluedo.

Question

1. D'après l'examen du programme, combien de symboles de propositions sont-ils définis ? Combien de phrases la base de connaissances contient-elle ?
2. Après exécution du programme avec la méthode de vérification de modèles (*model checking*), quelle est la combinaison de cartes trouvées satisfaisant la base de connaissance ?

2 Résolution d'énigmes

En 1978, le logicien Raymond Smullyan a publié « *What is the name of this book ?* », un livre d'énigmes logiques. Parmi les énigmes du livre figurait une catégorie d'énigmes que Smullyan appelait « *Knights and Knaves* » (chevaliers et valets).

Dans ces énigmes, les informations suivantes sont données :

— Chaque personnage est soit un chevalier, soit un valet.

— Un chevalier dira toujours la vérité : si le chevalier énonce une phrase, alors cette phrase est vraie. Inversement, un valet mentira toujours : si un valet énonce une phrase, cette phrase est fausse.

À partir d'un ensemble de phrases prononcées par chacun des personnages, l'objectif de l'énigme est de déterminer, pour chaque personnage, si ce dernier est un chevalier ou un valet.

Par exemple, prenons une énigme simple avec un seul personnage nommé A. A dit : « Je suis à la fois un chevalier et un valet ».

Logiquement, nous pourrions penser que si A était un chevalier, cette phrase devrait être vraie. Mais nous savons qu'il est impossible que cette phrase soit vraie, car A ne peut pas être à la fois chevalier et valet. Nous pouvons donc conclure que A doit être un valet.

L'objectif de cette partie du TP est de résoudre grâce à la logique des propositions quatre énigmes de ce type. Pour ce faire, compléter le programme puzzle.py fourni et trouver les solutions grâce à l'option par défaut `-c`.

Question

1. À partir de l'expression de chacune des énigmes en logique de proposition et en utilisant votre programme, énumérez les solutions trouvées.

3 Mise en œuvre de la résolution

Il vous est demandé dorénavant d'implémenter la seconde solution présentée en cours pour vérifier l'implication sémantique, à savoir l'application de règles d'inférence et de la résolution.

Cette mise en œuvre implique tout d'abord de convertir n'importe quelle expression de propositions en forme normale conjonctive (CNJ). La résolution permet ensuite de démontrer par l'absurde l'implication $KB \models \alpha$ en montrant que $KB \wedge \neg\alpha$ conduit à la proposition fausse à l'aide de l'algorithme suivant.

Function *resolution_inference(*KB, α*) : bool*
  input : KB : expression logique représentant la base des connaissances, *alpha* : expression logique à démontrer
  output: Vrai si α peut être démontré, Faux sinon
  clauses $\longleftarrow$ l'ensemble des clauses sous forme CNF représentant $KB \wedge \neg\alpha$;
  while *Vrai* **do**
    new_clauses $\longleftarrow$ {};
    foreach $c_i, c_j \in$ *clauses* **do**
      resolvents $\longleftarrow$ resolve(c_1, c_2);
      if *resolvents contient la clause vide* **then**
        Retourner Vrai;
      end
      new_clauses $\longleftarrow$ *new_clauses* $\cup$ *resolvents*;
    end
    if *new_clauses* $\subseteq$ *clauses* **then**
      Retourner Faux;
    end
    clauses $\leftarrow$ *clauses* $\cup$ *new_clauses*;
  end
end

Compléter le code de logic.py. Pour vous aider les parties les plus compliquées vous sont données.

Vous pourrez tester votre code avec test-resolution.py et l'option `-r` des programmes clue.py et puzzle.py.

Question

1. Même si les fonctions les plus complexes vous sont données, cette partie reste la plus longue du TP. Précisez clairement les fonctions dont vous avez pu tester le fonctionnement.